Chapitre 4 : La persistance des données

La **persistance des données** est un concept fondamental en informatique. Pour faire simple, c'est la capacité d'une application à conserver des informations de manière durable, même après que le programme a été arrêté ou que l'ordinateur a été éteint.

Sans persistance, toutes les données résideraient uniquement dans la **RAM** (mémoire vive) et disparaîtraient à la fermeture du logiciel.

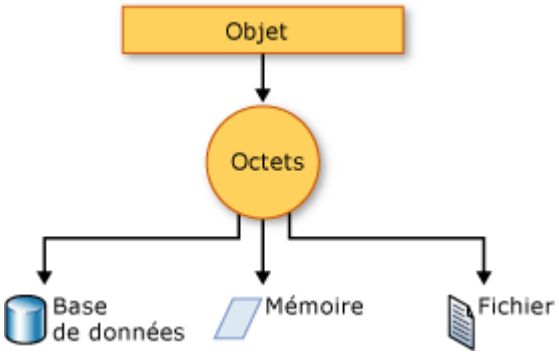
Les 3 piliers de la persistance

Pour qu'une donnée soit considérée comme "persistante", elle doit généralement passer par un processus de stockage sur un support non volatil (disque dur, SSD, Cloud).

1. **La Source (RAM) :** Les données sont créées et manipulées par votre code (variables, objets).
2. **Le Mécanisme de Transfert :** Un pont (comme une API, un ORM ou un pilote SQL) qui envoie les données vers le stockage.
3. **Le Support de Stockage :** Le lieu physique ou logique où la donnée est sauvegardée.

Il y a, aujourd'hui, quatre manières d'assurer la persistance des objets, qui, par sophistication croissante, sont : la simple écriture et lecture des valeurs d'attributs à sauvegarder sur un fichier, la sérialisation des objets, l'interaction avec une base de données relationnelle, et l'utilisation de bases de données orientées objet. Nous pourrions également les distinguer en fonction de leur respect de la nature « objet » des données à sauvegarder, leur facilité d'emploi, leur diffusion. Nous allons passer en revue ces quatre manières, leurs défauts, leurs qualités et l'état actuel de cette diffusion, en les appliquant à la sauvegarde d'un objet prédateur.

Cette illustration affiche le processus global de la persistance.



L'objet est sauvegardé par un flux qui contient non seulement les données, mais également des informations sur le type d'objet, notamment sa version, sa culture et son nom d'assembly. À partir de ce flux, il peut être stocké dans une base de données, dans un fichier ou en mémoire.

- **Simple sauvegarde sur fichier Utilisation des « streams »**

Tous les langages de programmation permettent, par l'utilisation de « streams » (traduction « flux »), et de manière assez semblable, d'écrire et de lire des octets sur un fichier. Un « stream » est un tube

à octet, qui connecte le programme en train de s'exécuter et tout type de périphérique, disque et autres (réseau, imprimantes…). Cette abstraction permet de traiter tous les périphériques comme un seul. Comme à l'état primitif le « stream » ne fait circuler que des octets, cette classe est souvent sujette à des spécifications par le jeu de l'héritage, ou à des emboîtements (en Java et en C#) par lesquels on spécialise, par un second « stream », connecté à même le premier, un mode d'écriture ou de lecture sur le périphérique.

Qui sauve quoi ?

Effectuons, dans la suite, la sauvegarde de quatre données concernant chaque prédateur, son type, sa vitesse en x et en y, et son énergie, dans un fichier texte appelé leFichierPredateur.txt.. Nous supposerons, et c'est plutôt une bonne pratique, que le prédateur soit responsable de la persistance de ses données. Qui, mieux que lui, pourrait s'en occuper, puisque lui seul y accède de la manière la plus directe qui soit. Deux méthodes sont rajoutées dans la classe Predateur, que nous nommons *sauveDonnees()*, responsable de l'écriture sur le disque, et *litLesDonnees()* responsable de la lecture. Ce sont ces deux méthodes dont le code sera récrit en fonction des mécanismes de persistance mis en œuvre.

C'est le point crucial qui sépare un "code qui marche" d'un "code professionnel". En architecture logicielle, on parle souvent du principe **SRP (Single Responsibility Principle)**.

Voici comment notre code respecte ce principe en séparant les responsabilités :

- **Le Prédateur : L'Expert de l'Objet**

Sa responsabilité : Définir ce qu'est un être vivant et protéger ses données.

- **Il sauve quoi ?** Ses propres caractéristiques internes (Vitesse, Énergie).
- **Pourquoi ?** Parce qu'il est le seul à avoir accès à ses données private. Si tu changes la façon dont l'énergie est calculée, tu ne modifies que cette classe.
- **Il ne sait pas :** Qu'il y a un fichier sur le disque ou qu'il fait partie d'une liste.

- **La Jungle : Le Gestionnaire de Collection**

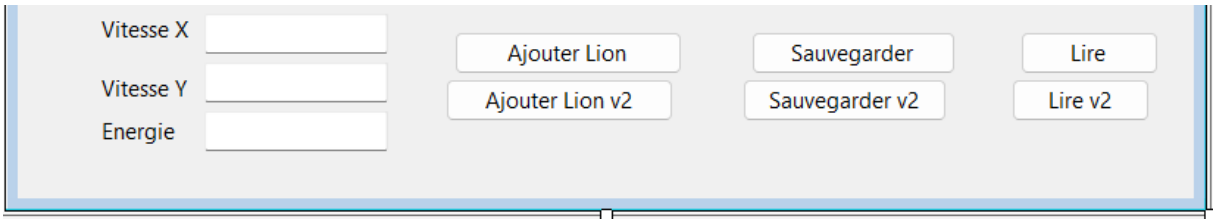
Sa responsabilité : Organiser la vie de groupe et la logistique.

- **Elle sauve quoi ?** La structure de groupe (la List). Elle gère l'ouverture du "contenant" (le fichier JSON).
- **Pourquoi ?** Parce qu'un lion ne devrait pas avoir le pouvoir d'ouvrir ou de fermer un fichier global ; c'est le rôle du gestionnaire de l'environnement.
- **Elle ne sait pas :** Comment le prédateur traite ses données privées. Elle se contente de lui passer son morceau de JSON.

- **L'Interface (Boutons) : Le Messager**

Sa responsabilité : Faire le pont entre l'humain et la machine.

- **Elle sauve quoi ?** Rien. Elle transforme des clics en ordres.
- **Pourquoi ?** Pour que ton code puisse être réutilisé. Si demain tu veux transformer ton appli Windows en appli Mobile, tu gardes Jungle et Predateur intacts, tu ne changes que les boutons.




```
using System; using System.IO;
public class Predateur
{
    // Champs de la classe
    public int vitX, vitY;
    public double energie;

    public void setVitesse(int x, int y) { vitX = x; vitY = y; }
    public void setEnergie(double e) { energie = e; }

    public void sauveDonnees(StreamWriter maSortie)
    {
        maSortie.WriteLine("---Predateur---"); // Marqueur de début
        maSortie.WriteLine(vitX);
        maSortie.WriteLine(vitY);
        maSortie.WriteLine(energie);
    }

    public void litLesDonnees(StreamReader monEntree)
    {
        monEntree.ReadLine(); // Saute le marqueur
        int vX = int.Parse(monEntree.ReadLine());
        int vY = int.Parse(monEntree.ReadLine());
        double e = double.Parse(monEntree.ReadLine());

        setVitesse(vX, vY);
        setEnergie(e);
    }
}
public class Jungle
 {
     public List<Predateur> lesLions = new List<Predateur>();
     private const string NomFichier = "jungle_data.txt";

     // Ajoute un lion à la jungle
     public void AjouterLion(int x, int y, double e)
     {
         var lion = new Predateur();
         lion.setVitesse(x, y);
         lion.setEnergie(e);
         lesLions.Add(lion);
     }

     public void Sauvegarder()
     {
         try
         {
```

```
            // "using" assure que le fichier est fermé même en cas d'erreur
            using var writer = new StreamWriter(NomFichier);
            foreach (var lion in lesLions)
            {
                lion.sauveDonnees(writer);
            }
            Console.WriteLine($"Succès : {lesLions.Count} lions sauvegardés.");
        }
        catch (IOException ex)
        {
            Console.WriteLine($"Erreur d'écriture : {ex.Message}");
        }
    }

    public void Charger()
    {
        if (!File.Exists(NomFichier)) return;

        try
        {
            using var reader = new StreamReader(NomFichier);
            lesLions.Clear(); // On vide la liste actuelle avant de charger

            while (!reader.EndOfStream)
            {
                var lion = new Predateur();
                lion.litLesDonnees(reader);
                lesLions.Add(lion);
            }
            Console.WriteLine("Chargement terminé.");
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Erreur lors du chargement : {ex.Message}");
        }
    }
}
```

```
    public partial class Form1 : Form
    { private Jungle maJungle = new Jungle();
        private void button1_Click(object sender, EventArgs e)
        {
            var lion = new Predateur();
            lion.setVitesse(int.Parse(txtVitX.Text), int.Parse(txtVitY.Text));
            lion.setEnergie(double.Parse(txtEnergie.Text));

            maJungle.lesLions.Add(lion);
            MessageBox.Show("Lion ajouté en mémoire !");
        }

        private void button2_Click(object sender, EventArgs e)
        {
            try
            {
                maJungle.Sauvegarder();
                MessageBox.Show("Fichier enregistré avec succès !");
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur : " + ex.Message);
            }
        }

        private void button3_Click(object sender, EventArgs e)
```

```
{
    maJungle.Charger();
    // Exemple pour afficher le nombre de lions chargés
     MessageBox.Show($"Lions chargés : {maJungle.lesLions.Count}");
}      // On convertit le texte des TextBox en nombres
```

- **La sérialisation**

Bien que cette pratique nous assure que les informations à sauvegarder le seront effectivement, elle présente l'inconvénient de « sonner faux » dans une perspective plus OO. En effet, nul cas n'est fait de l'organisation en objet des informations à sauvegarder. Qu'ils soient contenus dans des objets ou non, ces entiers, doubles ou caractères à sauvegarder, le seront, d'une seule et même manière.

De plus, nous savons que les objets sont connectés entre eux, par un réseau relationnel qui peut, très vite, devenir important et complexe. Rien n'est prévu pour que la sauvegarde d'un des nœuds de ce réseau puisse déboucher sur la sauvegarde du réseau dans son entièreté. C'est au programmeur de penser la persistance, de manière à conserver également les liens qui relient les objets entre eux, lors de leur sauvegarde. L'utilisation de simples tubes à octet connectés au fichier ne le permet pas.

Si vous sauvegardez un attribut de type référent, vous vous limitez à sauvegarder une adresse, sans que l'objet adressé par cette adresse ne le soit également à un endroit correspondant sur le disque dur. En substance, il est nécessaire ici de recourir à un mode de sauvegarde plus puissant, qui permettrait, très facilement, de reproduire sur le disque dur une copie des objets et de leur structure d'interconnexion, c'est-à-dire de faire, à un instant donné, une copie miroir du tissu relationnel des objets qui se trouve dans la RAM. La lecture d'un des objets du réseau suffirait, pareillement, à la reproduction dans la RAM du réseau dans son entièreté. Ce mécanisme porte le nom de «sérialisation».

Il suffit de découvrir la nouvelle version des méthodes sauveDonnees()et litLesDonnees()pour se convaincre de l'extrême simplicité de la mise en œuvre de la sérialisation.

```
[Serializable]
 public class PredateurV2
 {
     // Données publiques
     public int VitX { get; set; }
     public int VitY { get; set; }

     // Donnée privée accessible uniquement via l'attribut JsonInclude
     [JsonInclude]
     private double _energie;

     // Constructeur vide obligatoire pour la désérialisation
     public PredateurV2() { }

     // Méthode pour initialiser un prédateur (puisque _energie est privée)
     public void Init(int x, int y, double e)
     {
         VitX = x;
         VitY = y;
         _energie = e;
     }

     // --- LOGIQUE DE LECTURE (AUTO-RECONSTITUTION) ---
     // Elle prend un STRING en paramètre, plus un StreamReader !
     public void litLesDonnees(string fragmentJson)
     {
         if (string.IsNullOrWhiteSpace(fragmentJson)) return;

         // On crée un objet temporaire à partir du texte reçu
         var source = JsonSerializer.Deserialize<PredateurV2>(fragmentJson);
```

```
        if (source != null)
        {
            // On recopie les données dans l'instance actuelle (this)
            this.VitX = source.VitX;
            this.VitY = source.VitY;
            this._energie = source._energie;
        }
    }
}
    public class JungleV2
    {
        public List<PredateurV2> lesLions = new List<PredateurV2>();
        private const string NomFichier = "ma_jungle.json";
        // --- CETTE MÉTHODE MANQUAIT ---
        public void Ajouter(PredateurV2 p)
        {
            if (p != null)
            {
                lesLions.Add(p);
            }
        }

        // Utile pour afficher le compte sur l'interface
        public int GetNbLions() => lesLions.Count;
        // --- SAUVEGARDE GLOBALE ---
        public void SauvegarderTout()
        {
            try
            {
                var options = new JsonSerializerOptions { WriteIndented = true };
                // On sauve la liste complète. Le moteur JSON ira lire les
                // propriétés et les champs [JsonInclude] de chaque lion.
                string jsonGlobal = JsonSerializer.Serialize(lesLions, options);
                File.WriteAllText(NomFichier, jsonGlobal);

                MessageBox.Show("Sauvegarde réussie !");
            }
            catch (Exception ex)
            {
                MessageBox.Show("Erreur de sauvegarde : " + ex.Message);
            }
        }

        // --- CHARGEMENT GLOBAL ---
        public void ChargerTout()
        {
            if (!File.Exists(NomFichier)) return;

            try
            {
                string jsonBrut = File.ReadAllText(NomFichier);

                // 1. On découpe le JSON global en une liste d'éléments JSON
individuels
                var elements =
JsonSerializer.Deserialize<List<JsonElement>>(jsonBrut);

                if (elements != null)
                {
                    lesLions.Clear(); // On vide la liste avant de recharger

                    foreach (JsonElement item in elements)
```

```
                {
                    PredateurV2 lion = new PredateurV2();

                    // 2. On transforme l'élément structuré en TEXTE (string)
                    string lionJson = item.GetRawText();

                    // 3. On demande au lion de se charger lui-même
                    lion.litLesDonnees(lionJson);

                    lesLions.Add(lion);
                }
            }
            MessageBox.Show($"{lesLions.Count} lion(s) chargé(s) !");
        }
        catch (Exception ex)
        {
            MessageBox.Show("Erreur de chargement : " + ex.Message);
        }
    }

    private void button5_Click(object sender, EventArgs e)
    {
        // On appelle simplement la méthode de la jungle
        // C'est elle qui connaît le nom du fichier et la logique JSON
        maJungleV2.SauvegarderTout();
    }

    private void button4_Click(object sender, EventArgs e)
    {

        // On demande à la jungle de se charger
        maJungleV2.ChargerTout();

        // Optionnel : Afficher un message de confirmation personnalisé
        // ou mettre à jour un compteur sur ton interface
        MessageBox.Show("Nombre de lions : " + maJungleV2.lesLions.Count);

    }

    private void AjoutV2_Click(object sender, EventArgs e)
    {

        try
        {
            // 1. Création d'un nouveau prédateur
            PredateurV2 nouveauLion = new PredateurV2();

            // 2. Récupération des données depuis l'interface
            // Remplace txtVitX, txtVitY et txtEnergie par les noms de tes
contrôles
            int vx = int.Parse(txtVitX.Text);
            int vy = int.Parse(txtVitY.Text);
            double en = double.Parse(txtEnergie.Text);

            // 3. Initialisation et ajout à la jungle
            nouveauLion.Init(vx, vy, en);
            maJungleV2.Ajouter(nouveauLion); // Assure-toi d'avoir une
méthode Ajouter dans Jungle

            MessageBox.Show("Le lion a été ajouté à la liste temporaire.");

            // Optionnel : vider les champs après ajout
            txtVitX.Clear(); txtVitY.Clear(); txtEnergie.Clear();
```

```
            }
            catch (Exception ex)
            {
                MessageBox.Show("Veuillez saisir des nombres valides ! " +
ex.Message);
            }
        }
```

La présence de [Serializable] devant la classe déclare la classe sérialisable, ainsi que la présence de [NonSerialized] devant un attribut, indiquant que la sérialisation s'arrête à celui-ci. Par défaut on sérialise les attribut public mais on utilise `[JsonInclude] pour les privte`. La lecture et l'écriture se font au moyen des méthodes Serialize et Deserialize et, lors de la lecture, il est nécessaire de « caster » l'objet lu dans la classe d'accueil.
Notez qu'en C# et .Net en général, il est également possible de sérialiser très facilement des objets dans le format XML, en substituant simplement le `JsonSerializer` par l'utilisation de la classe XMLSerialize et des méthodes Serialize et Deserialize recevant des « streams » en argument.

La simplicité du mécanisme de sérialisation dissimule une grande puissance et une formidable efficacité. Il suffit de comparer déjà les deux versions programmées de la persistance pour s'en rendre compte. Ce mécanisme rend parfaitement justice à la nature OO des données à sauver. De plus, il suffit de partir d'un nœud du réseau d'interaction entre objets pour qu'automatiquement, tous les autres soient pris en charge, en lecture comme en écriture.

En outre, il existe aujourd'hui un mode de dépôt des données sur disque dur, mode très lisible, optimisé, très efficace, et surtout qui s'est extraordinairement répandu : les bases de données. Pour couronner le tout, ces bases de données ressemblent comme deux gouttes d'eau au modèle OO. Hélas, des gouttes qui restent très troubles, car si l'équivalence table/classe et enregistrement/objet est plus que tentante, elle est loin d'être vérifiée.

Exercice

Ecrire une classe sérializable livre en utilisant le `JsonSerializer` puis en utilisant la classe XMLSerialize.

```
using System.Xml.Serialization ;

using System.IO ;
public class XMLWrite
{
static void Main(string[] args)
{
Book overview = new Book();
overview.title = "Serialization Overview";

overview.WriteXML();
overview.ReadXML() ;
}
}

public class Book
{
public String title;
public void WriteXML()
{
      XmlSerializer writer = new XmlSerializer(typeof(Book));

      var path = "livre.xml";
      FileStream file = File.Create(path);

      writer.Serialize(file, this);
      file.Close();
}


        Public static Book ReadXML()
        {
                XmlSerializer reader = new XmlSerializer(typeof(Book));
                StreamReader file = new StreamReader("livre.xml");
                Book        bb        =        (Book)reader.Deserialize(file);
                Console.WriteLine(bb.title);
                Return bb;
        }
}
```

- **ORM avec Entity Framework**

….